

C++ Programming Textbook Notes

Module: Method Overriding, Base Class Pointers, and Runtime Polymorphism

1. METHOD OVERRIDING MECHANICS

OOP Core

Conceptual Explanation

Method Overriding occurs in C++ Object-Oriented Programming when a derived (child) class redefines a member function that is already defined in its base (parent) class. The overridden function in the child class must share the exact same name, same parameter list (arguments), and same return type—collectively called the function **signature**.

Unlike method overloading (which happens within the same scope across different signatures), method overriding operates across an inheritance hierarchy to allow specialized derived behavior while maintaining a uniform base interface.

Method Overriding vs. Function Overloading

Feature	Function Overloading	Method Overriding
Scope	Same class or global scope	Across Base and Derived classes (Inheritance)
Function Signature	Must differ in parameters (count or types)	Must be identical (name, parameters, return type)
Binding Type	Static / Compile-Time Binding	Dynamic / Runtime Binding (with <code>virtual</code>)
Inheritance	Not required	Mandatory inheritance relationship

Syntax & Golden Rules

- **Rule 1: Identical Signature:** The name, parameter types, and constness must match exactly.
- **Rule 2: Public/Protected Inheritance:** Overriding requires derived class access to the base method.
- **Rule 3: C++11 `override` Specifier:** Always tag overridden methods with `override` so the compiler explicitly verifies matching base signatures.

Code Example: Basic Overriding without Virtual Keywords

```
#include <iostream> using namespace std; class Parent { public: void show() { cout << "Th
```

Static Binding Gotcha: Calling `obj.show()` directly using a derived object variable executes `Child::show()`. However, if assigned to a `Parent` object variable or accessed via non-virtual base pointers, static binding defaults to `Parent::show()`!

The Architectural Problem

When we instantiate distinct derived objects directly (e.g., `Child obj; obj.show();`), method resolution is tied rigidly to the concrete variable type. Creating separate object types for every derived class in large systems is inflexible, leads to redundant code, and breaks extensible design patterns.

Core Question: Why use Base Class Pointers?

If we can invoke derived class functions directly via concrete child objects, why do we need base class pointers?

Answer: To achieve runtime flexibility! Base class pointers allow a single uniform interface to manipulate any derived object dynamically at runtime without knowing its concrete class in advance.

Real-World Analogy: The Universal Remote Control

Consider a modern home entertainment setup with multiple electronic appliances:

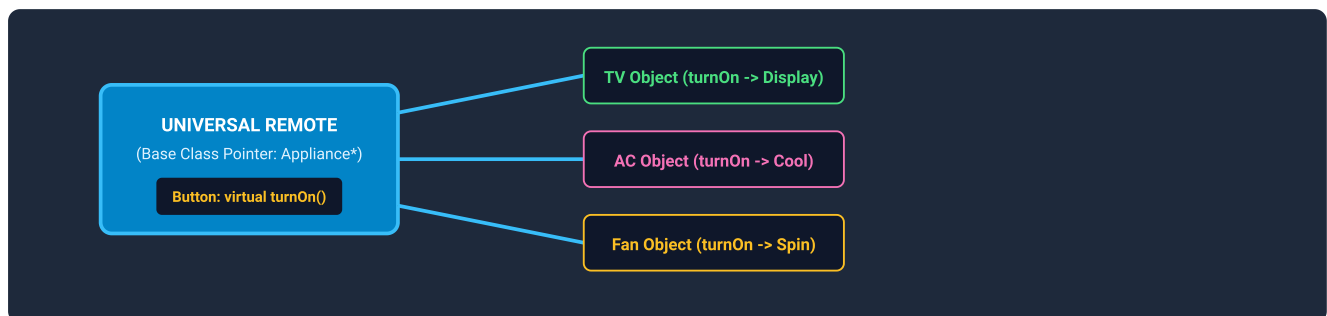
Without Base Pointers (Rigid System)

You need a physical separate remote control for every single device: one remote for TV, one remote for AC, and another for Fan. Your software code must manage 10 different object types independently.

With Base Pointer & Virtual Functions

You hold **ONE Universal Remote Control** (Parent Pointer). It features a generic button labeled `POWER()` (Virtual Function). Pressing the button automatically triggers the correct behavior on whichever appliance (TV/AC/Fan) is targeted.

Universal Remote Control Structural Diagram



Key Insight

You press **ONE** button on the remote, and the correct device behavior executes automatically. You do not need to rewrite remote logic whenever a new appliance class is added.

```
// Illustrating Static Binding Failure without 'virtual' Parent* ptr; Child childObj; ptr
```

Virtual Functions: Enabling Dynamic Dispatch

A **Virtual Function** is a member function declared inside a base class using the `virtual` keyword that is intended to be overridden in derived classes.

When a base class function is marked `virtual`, the compiler postpones function call resolution from compile time to runtime—a mechanism known as **Dynamic Binding (Late Binding)**.

Static Binding vs. Dynamic Binding

Property	Static Binding (Early Binding)	Dynamic Binding (Late Binding)
Resolution Time	Determined at Compile Time	Determined at Run Time
Mechanism	Direct call based on pointer/reference type	Indirect call via vtable pointer lookup
Performance	Slightly faster (no pointer dereferencing)	Minimal runtime overhead for vtable lookups
Keyword Required	Default C++ behavior (None)	Requires <code>virtual</code> keyword in Base class

Syntax & Rules for Virtual Functions

- Keyword Placement:** Declared using `virtual` inside the Base class declaration.
- Inherited Virtual Status:** Once a function is declared `virtual` in a base class, it remains virtual across all subsequent derived classes automatically (though specifying `override` in derived classes is recommended).
- Virtual Destructors:** Base classes with virtual functions MUST declare a `virtual ~Base()` destructor to ensure derived object cleanup avoids memory leaks.

Complete Program: Runtime Polymorphism Implementation

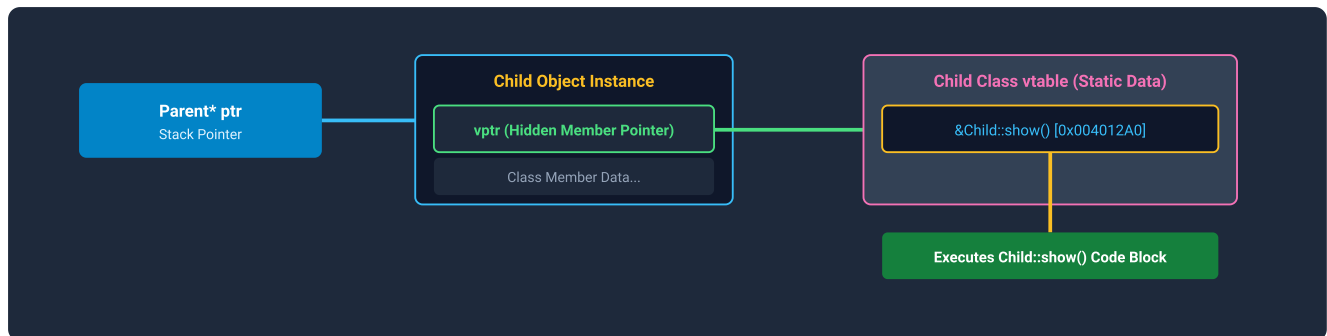
```
#include <iostream> using namespace std; class Parent { public: // Marking function as virtual
```

Under the Hood Mechanics

How does C++ determine which overridden function to call at runtime? The compiler achieves dynamic dispatch using two internal structures:

- **vtable (Virtual Table):** A static array of function pointers created by the compiler for every class that contains or inherits virtual functions. Each entry in the table stores the memory address of the corresponding virtual function.
- **vptr (Virtual Pointer):** A hidden pointer automatically added as the first member variable of every class instance containing virtual functions. It points directly to the class's static vtable in memory.

Detailed RAM Memory Layout Diagram



Execution Sequence during ptr->show() Call

1. **Step 1 (Dereference Pointer):** The program dereferences ptr to reach the target Child object memory.
2. **Step 2 (Fetch vptr):** It reads the hidden vptr stored inside the object to locate the class vtable.
3. **Step 3 (Table Index Lookup):** It indexes into the vtable to retrieve the function address corresponding to show().
4. **Step 4 (Indirect Jump):** It jumps to address 0x004012A0 and executes Child::show().

Memory Footprint Overhead: Adding a virtual function to a class increases each object's physical size by the size of one pointer (e.g., 8 bytes on a 64-bit architecture) to accommodate the hidden vptr.

Common Gotchas & Pitfalls

1. Non-Virtual Destructors in Base Classes: If a derived object created via `new Child()` is deleted using a base pointer (`delete ptr;`) without a virtual base destructor, only `Parent::~~Parent()` executes, causing resource memory leaks in the child!

2. Signature Mismatch / Function Hiding: If parameters differ slightly (e.g., `void show(int x)` in child vs. `void show()` in parent), the child method hides the base method instead of overriding it.

3. Slicing Problem: Assigning a derived object directly to a base value object (e.g., `Parent p = childObj;`) slices off the child attributes and strips away dynamic polymorphism.

Best Practices for Modern C++

- **Use `override` explicitly:** Always tag overridden functions with `override` (C++11 standard). This forces the compiler to report errors if the signature does not match a base virtual function.
- **Always define Virtual Destructors:** Mark base class destructors as `virtual ~Base() = default;` whenever virtual functions are present.
- **Use `final` keyword when needed:** Tag functions or classes with `final` if you want to prevent further overriding down the inheritance tree.

Complete Summary Cheat Sheet

Scenario	Pointer Type	Object Type	Virtual Used?	Method Executed
Direct Call	Child	Child	No	Child::show()
Base Pointer	Parent*	Child	No	Parent::show() (Static)
Base Pointer	Parent*	Child	Yes	Child::show() (Dynamic)

Final Takeaway

Runtime polymorphism in C++ is achieved using **Base Class Pointers** pointing to derived objects combined with **Virtual Functions**. This empowers developers to build modular, flexible, and scalable software systems.